Project Report: Controlling LEDs with Your Fingers

Computer vision (MediaPipe) + IoT messaging (MQTT) + a microcontroller (ESP32)

This report explains the whole project from the ground up. It assumes no background in computer vision, networking or electronics. Technical words appear in **bold** the first time and are collected in the glossary at the end.

Table of contents

1. What the project does
2. The big picture
3. How a computer sees: images and video
4. MediaPipe: finding the hand
5. From 21 points to 5 fingers
6. MQTT: sending the result
7. The ESP32 and the LEDs
8. Code walkthrough
9. Real-time behaviour and timing
10. Corrections made to the first version
11. Limitations and possible improvements
12. Security note
13. Glossary

---

1. What the project does

A person stands in front of a webcam and raises some fingers. On a small electronic board a few meters away, five LEDs (one per finger) turn on or off to match, instantly:

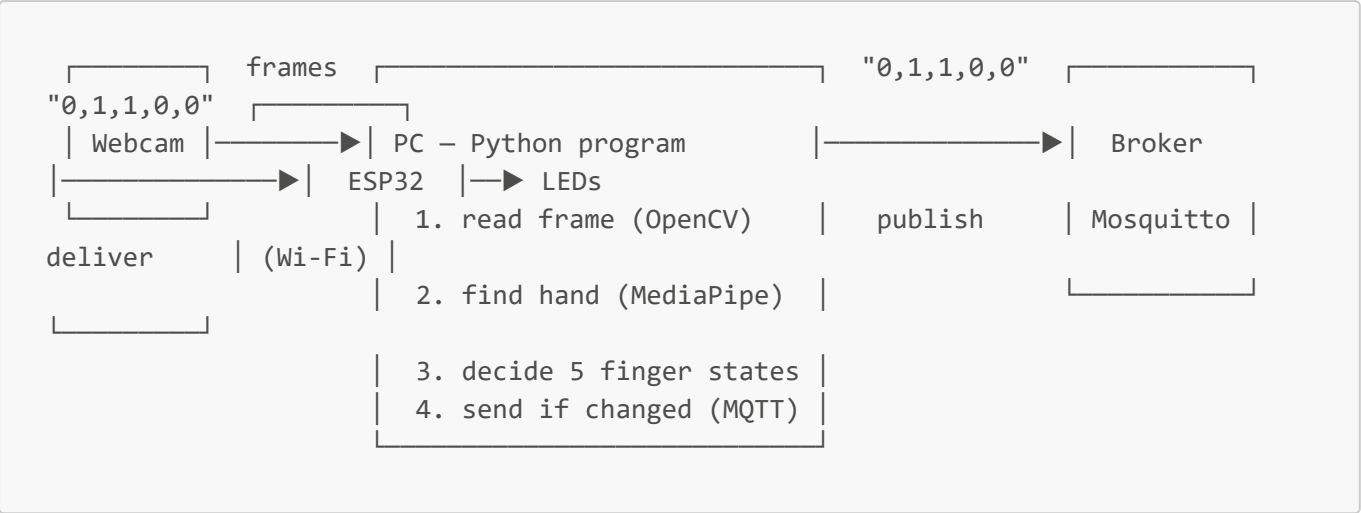
Hand	Message	LEDs (thumb → pinky)
Open hand 🖐	`1,1,1,1,1`	💡 💡 💡 💡 💡
Fist ✊	`0,0,0,0,0`	⚫ ⚫ ⚫ ⚫ ⚫
Peace ✌	`0,1,1,0,0`	⚫ 💡 💡 ⚫ ⚫
Thumbs up 👍	`1,0,0,0,0`	💡 ⚫ ⚫ ⚫ ⚫

The project combines three separate fields:

- **Computer vision**: a program that looks at camera images and understands that there is a hand and where each finger is.
- **Networking / IoT**: a way to send that information over Wi-Fi to another device.
- **Embedded electronics**: a tiny computer (the ESP32) that receives the information and switches real lights on and off.

---

2. The big picture



The journey of one finger movement:

1. You raise your index finger.
2. The webcam captures a new picture (a **frame**), about 30 times per second.
3. The Python program gives that picture to **MediaPipe**, which returns the positions of 21 points on your hand.
4. The program compares a few of those points and concludes "index = raised". The new state is `0,1,0,0,0`.
5. This differs from what was last sent, so the program **publishes** the text `0,1,0,0,0` to the **MQTT broker** under the name (**topic**) `hand/gesture`.
6. The broker immediately forwards it to every device **subscribed** to `hand/gesture`, which here is the ESP32.
7. The ESP32 reads the 5 digits and sets the voltage on 5 pins. Pin 26 (index) goes to 3.3 V and its LED lights up.

All of this takes roughly a tenth of a second.

There are three programs on two devices, plus one "post office":

Part	Runs on	Language	File(s)
Finger detector	PC	Python	`finger_detector/*.py`
MQTT broker	PC (could be any computer on the network)	Mosquitto (ready-made program)	`mosquitto.conf`
LED controller	ESP32	C++ (Arduino)	`esp32/hand_leds/hand_leds.ino`

3. How a computer sees: images and video

3.1 An image is a grid of numbers

A digital image is a rectangle of tiny colored squares called **pixels**. A webcam image is typically 640 × 480 pixels, which is 307,200 pixels. Each pixel's color is stored as three numbers from 0 to 255, for the amount of **red**, **green** and **blue** light:

- `(255, 0, 0)` = pure red, `(0, 0, 0)` = black, `(255, 255, 255)` = white.

For the computer, "an image" is therefore just a big table of numbers: 480 rows × 640 columns × 3 colors.

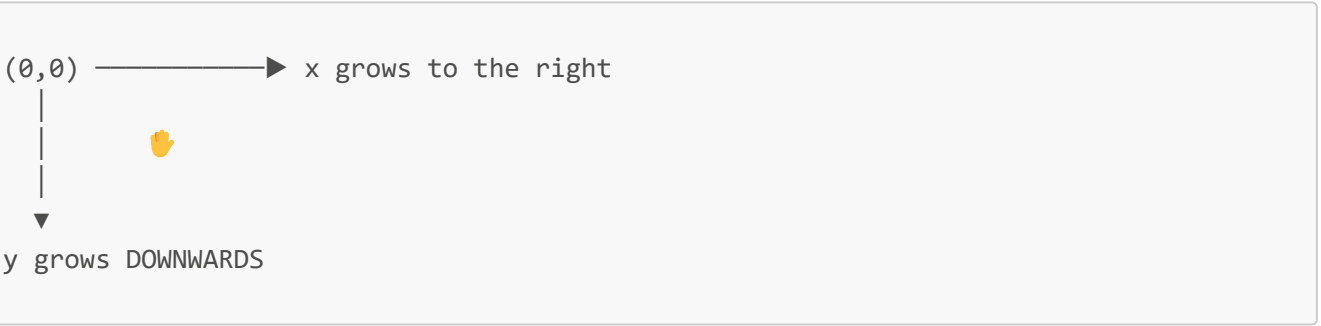
> **BGR vs RGB.** OpenCV, the library we use to read the camera, stores the three numbers in the order **Blue, Green, Red**. MediaPipe expects **Red, Green, Blue**. That's why the code converts the image with `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)` before giving it to MediaPipe. If you skip this, colors are swapped and detection gets worse.

3.2 Video is many images per second

A video is a sequence of images (frames) shown quickly one after another. A webcam usually delivers **30 frames per second (FPS)**, so a new frame arrives every ~33 milliseconds. Our program processes each frame in a loop: read → analyse → (maybe) send → show → repeat.

3.3 Image coordinates

Positions in an image are measured from the **top-left corner**:



This matters later: a finger pointing **up** has a **smaller** y value at its tip than at its base.

3.4 Mirroring

The program flips each frame horizontally (`cv2.flip(frame, 1)`). The video then behaves like a mirror: you raise your right hand and the hand on the right side of the screen moves. People find this natural. It's also what MediaPipe assumes when it labels a hand "left" or "right".

3.5 OpenCV

OpenCV (Open Source Computer Vision Library) is the standard toolbox for working with images in Python. In this project it is used for the "plumbing" only: opening the webcam, reading frames, flipping, converting colors, drawing text and lines, and showing the window. It doesn't do the clever part. That's MediaPipe.

---

4. MediaPipe: finding the hand

4.1 What MediaPipe is

MediaPipe is a free library from Google with ready-to-use **machine learning models** for common vision tasks: detecting faces, body poses, hands, objects and more. We use its **Hand Landmarker**.

A **machine learning model** is a program that was not written rule by rule. Instead, it *learned* by looking at a huge number of example images. People marked where the hand joints are in tens of thousands of hand photos, and the model was trained to reproduce those marks on images it has never seen. The result is a file (`hand_landmarker.task`, about 8 MB) that contains the learned "knowledge". Our program downloads it automatically on the first run.

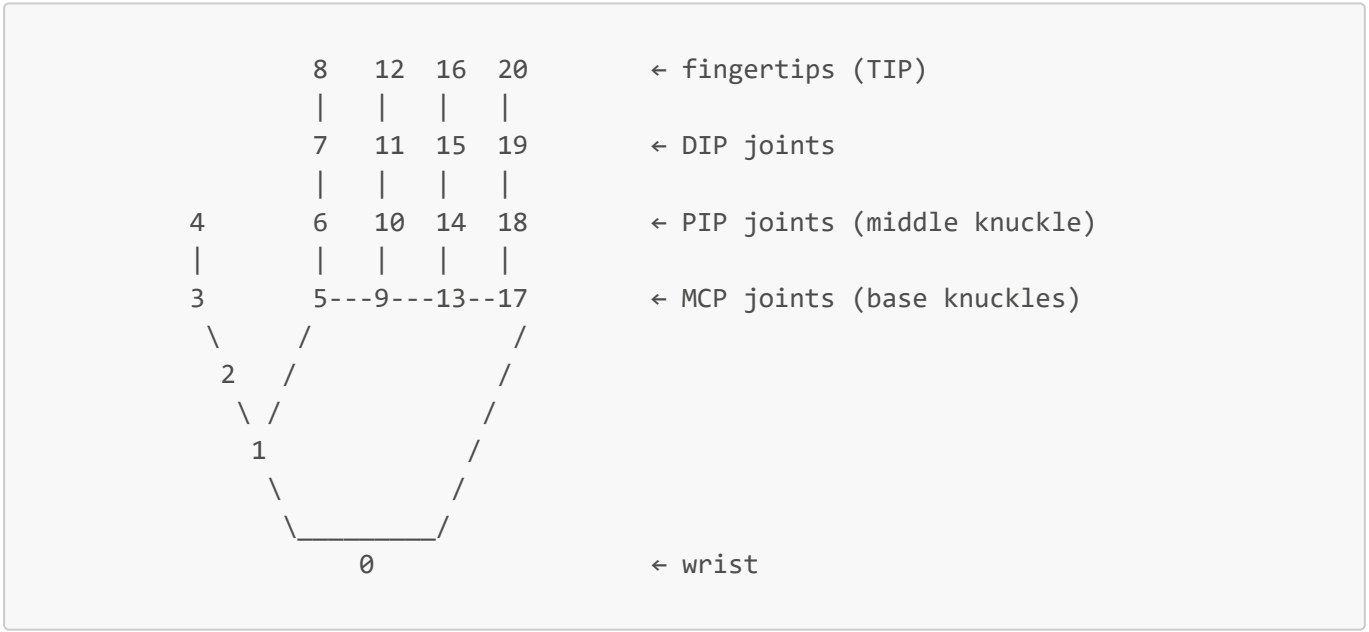
4.2 Two steps: find the palm, then the joints

The Hand Landmarker works in two stages:

1. **Palm detector**: scans the whole image and answers "is there a hand, and where is it?". It returns a box around the hand. Searching the whole image is the expensive part.
2. **Landmark model**: looks only inside that box and finds **21 precise points** (**landmarks**) on the hand.

In **VIDEO mode**, which is what we use, MediaPipe is smart: once it has found the hand, it doesn't rerun the palm detector on every frame. It assumes the hand is near where it was in the previous frame and only reruns the landmark model there. This is called **tracking**, and it's why detection is fast enough for real time. The palm detector only runs again when tracking loses the hand. VIDEO mode requires each frame to have an increasing **timestamp**, so the code passes one with every frame.

4.3 The 21 landmarks



Index	Point	Index	Point
0	Wrist		
1	Thumb base (CMC)	2	Thumb MCP
3	Thumb IP joint	4	**Thumb tip**
5	Index MCP (base)	6	**Index PIP**
7	Index DIP	8	**Index tip**

Index	Point	Index	Point
9–12	Middle finger (MCP, PIP, DIP, tip)	13–16	Ring finger
17–20	Pinky (MCP, PIP, DIP, tip)		

For each point, MediaPipe gives:

- **x**: horizontal position, from 0.0 (left edge) to 1.0 (right edge)
- **y**: vertical position, from 0.0 (top) to 1.0 (bottom)
- **z**: approximate depth, i.e. how close to the camera (not used here)

These are **normalized coordinates**: they are fractions of the image size, not pixels. So the same logic works whatever the camera resolution.

4.4 Confidence thresholds

The model doesn't just say "hand"; it says "hand, 93% sure". The settings `DETECTION_CONFIDENCE` and `TRACKING_CONFIDENCE` (0.7 = 70%) set the minimum certainty:

- **Higher** (e.g. 0.8): fewer false alarms, but it may lose the hand in poor lighting.
- **Lower** (e.g. 0.5): finds the hand more easily, but may "see" hands in random objects.

---

5. From 21 points to 5 fingers

MediaPipe tells us *where* the joints are. It doesn't say "the index finger is raised". We decide that with simple geometry.

5.1 The four fingers (index, middle, ring, pinky)

A finger is **raised** if its **tip** is higher in the image than its **middle knuckle (PIP)**:

```
raised            folded
  ●  tip           ┌─●  PIP
  │                │
  ●  PIP           ●  tip  (tip is now BELOW the PIP)
  │
```

Because y grows downwards, "higher" means a **smaller y**:

```
raised = landmarks[tip].y < landmarks[pip].y
```

The pairs used are (8, 6) for the index, (12, 10) for the middle, (16, 14) for the ring and (20, 18) for the pinky.

5.2 The thumb: the tricky one

The thumb doesn't fold *down* like the other fingers. It folds *sideways*, across the palm. So the "is the tip above the joint" test doesn't work for it. Instead we check whether the thumb tip sticks **out to the side**, away from the palm.

The catch: "out to the side" is **left** for one hand and **right** for the other. It also flips when you show the back of your hand instead of the palm. The first version of the code only checked "is the tip to the left of the joint?", so it worked for one hand only.

The new code compares two directions:

1. **Palm direction**: from the pinky base (point 17) to the index base (point 5). This shows which side of the hand the thumb is on.
2. **Thumb direction**: from the thumb's IP joint (point 3) to the thumb tip (point 4).

If both point the same way (both left or both right), the thumb tip is moving *away* from the palm, so the thumb is open:

```
palm_direction  = landmarks[5].x - landmarks[17].x
thumb_direction = landmarks[4].x - landmarks[3].x
thumb_open = palm_direction * thumb_direction > 0   # same sign = same direction
```

Multiplying two numbers gives a positive result only when they have the same sign (+ × + or − × −). This one line therefore handles the left hand, the right hand, the palm and the back of the hand without needing to know which hand it is.

5.3 Removing flicker: the stabilizer

When a finger is half raised, the tip and the knuckle are almost at the same height. Tiny changes between frames can make the answer jump back and forth, `1, 0, 1, 0…`, and the LED would flicker.

The `StateStabilizer` fixes this: a new state is accepted only after it has been seen `STABLE_FRAMES` **frames in a row** (default 2, which is about 66 ms). A one-frame glitch is ignored. Setting it to 1 disables the stabilizer for the fastest possible reaction; 3 makes it steadier.

5.4 Output

The result is always 5 values, in a fixed order:

Position	1st	2nd	3rd	4th	5th
Finger	thumb	index	middle	ring	pinky
Variable	a	b	c	d	e

If no hand is visible, the result is `0,0,0,0,0` (all LEDs off).

---

6. MQTT: sending the result

6.1 The problem

The PC knows the finger states, but the ESP32 is a separate device on the Wi-Fi network. We need a simple, fast and reliable way to send it a short message many times per second, and to recover automatically if one side restarts.

6.2 What MQTT is

MQTT (Message Queuing Telemetry Transport) is a messaging **protocol**, a set of rules for exchanging messages, designed for small IoT devices. It is lightweight: a message carries only a few bytes of overhead, so even a tiny microcontroller handles it easily.

MQTT uses a **publish / subscribe** model through a central server called the **broker**.

Analogy: a newspaper office.

- The **broker** is the newspaper office.
- A **topic** is the name of a newspaper, e.g. `hand/gesture`.
- A **publisher** writes an article and hands it to the office: our Python program.
- A **subscriber** has a subscription to that newspaper and receives every new issue as soon as it's printed: our ESP32.

```
  Python program ──publish("hand/gesture", "0,1,1,0,0")──▶  BROKER  ──▶ ESP32
(subscribed to "hand/gesture")
                                                                  ──▶ (any
other subscriber, e.g. a phone app)
```

Advantages of this model:

- **The sender and receiver don't need to know each other.** The PC doesn't need the ESP32's IP address, and you can add a second ESP32 or a phone dashboard without changing the Python code.
- **Either side can restart** independently and simply reconnect to the broker.
- **One connection, kept open.** Unlike loading a web page, the connection stays open, so each message goes out immediately with almost no delay.

Mosquitto is a popular free broker. Here it runs on the PC, listening on network **port 1883**, the standard MQTT port.

6.3 The MQTT features we use

Feature	What it means	Why we use it
Topic `hand/gesture`	The "address" of the messages	The ESP32 subscribes to exactly this name
Payload `"1,0,1,0,0"`	The content of the message	9 characters, simple to read on the ESP32
QoS 0	"Quality of Service" level 0: send once, no confirmation	Fastest. If one message is lost, the next change fixes it

Feature	What it means	Why we use it
Retain	The broker *remembers* the last message on the topic	When the ESP32 (re)starts, it immediately receives the current state instead of waiting for the next finger movement
Last Will (LWT)	A message the broker publishes *for us* if our program disappears without saying goodbye	If the PC program crashes, the broker sends `0,0,0,0,0` and the LEDs turn off instead of staying stuck
Keep-alive (30 s)	Both sides exchange tiny "still here" messages	This is how the broker detects that a client has disappeared
Automatic reconnect	If the connection drops, the client retries every 1 to 10 s	The program never crashes because the broker is unavailable

6.4 Publishing only on change

The camera produces about 30 frames per second, but your fingers don't change 30 times per second. The first version sent a message **every frame**, which means 30 identical messages per second. The new version sends a message **only when the state changes**, so a few messages per second at most. LED reaction is just as fast, and the network, the broker and the ESP32 do much less work.

---

7. The ESP32 and the LEDs

7.1 What a microcontroller is

A **microcontroller** is a complete small computer on one chip: processor, memory and **input/output pins**. It runs one program, forever, as soon as it gets power. There's no operating system, screen or keyboard.

The **ESP32** (made by Espressif) is a popular, cheap microcontroller with **built-in Wi-Fi and Bluetooth**. It runs at 240 MHz, which is plenty for this project. We program it with the **Arduino IDE** in C++.

7.2 GPIO pins

GPIO (General-Purpose Input/Output) pins are the metal legs of the board that the program controls. Configured as an **output**, a pin can be:

- **HIGH**: 3.3 volts. Current flows and the LED turns **on**.
- **LOW**: 0 volts. No current flows and the LED turns **off**.

In code: `digitalWrite(25, HIGH);` turns on the LED on pin 25.

7.3 The LED and its resistor

An **LED** (Light-Emitting Diode) lights up when current flows through it in the right direction, from the long leg (+) to the short leg (−). An LED on its own barely limits the current. Connected directly to a pin, it would draw too much and could burn out the LED or the pin.

A **resistor** in series limits the current. **Ohm's law** gives the current:

```
current = (supply voltage − LED voltage) / resistance
        = (3.3 V − 2.0 V) / 220 Ω  ≈  0.006 A  =  6 mA
```

6 mA is bright enough for a standard red LED and well under what an ESP32 pin can safely deliver. Blue and white LEDs need about 3 V, so they will be dimmer at 3.3 V. That's normal.

7.4 How the Arduino program is organised

Every Arduino program has two main functions:

- `setup()` runs **once** at power-on: it configures the pins, tests the LEDs, connects to Wi-Fi and prepares MQTT.
- `loop()` runs **over and over**, thousands of times per second: it checks the connections and lets the MQTT library process incoming messages.

When a message arrives, the MQTT library (**PubSubClient**) calls our function `onMessage()`, which:

1. reads the payload character by character and keeps the `0`s and `1`s, so `"1,0,1,0,0"` becomes `{1, 0, 1, 0, 0}`;
2. **checks** that there are exactly 5 values and nothing unexpected. Invalid messages are ignored and reported on the Serial Monitor instead of producing random LED patterns;
3. writes each value to its pin: `digitalWrite(LED_PINS[i], value)`.

7.5 Robustness on the ESP32

- **Non-blocking MQTT reconnect**: if the broker is unreachable, the ESP32 retries every 2 seconds without freezing.
- **Wi-Fi loss**: LEDs are turned off and the ESP32 reconnects.
- **Startup self-test**: all LEDs flash for 0.5 s at boot, so a wiring mistake is visible immediately.
- **Unique client ID** built from the chip's serial number (MAC address): two ESP32s won't kick each other off the broker.
- **Settings in `secrets.h`**: the Wi-Fi password is kept out of the main code and out of git.

---

8. Code walkthrough

The Python program is split into four files, each with a single job. This is called **separation of concerns**: each file can be understood, tested and changed on its own.

```
main.py ──uses──▶ config.py          (settings)
        ──uses──▶ finger_detector.py (vision)
        ──uses──▶ mqtt_client.py     (network)
```

8.1 `.env` and `config.py`: settings

`.env` is a simple text file of `NAME=value` lines (broker IP, camera number, thresholds…). Keeping these out of the code means:

- you don't edit code to move the project to another network;
- private values (passwords) don't end up on GitHub, because `.env` is in `.gitignore` and only the template `.env.example` is shared.

`config.py` reads `.env` with the **python-dotenv** library and converts every value to the correct type: number, true/false or path. It gathers them all in a single read-only `Settings` object, so the rest of the program uses `settings.mqtt_host` instead of reading raw text.

8.2 `finger_detector.py`: vision

Element	Role
`ensure_model()`	Downloads `hand_landmarker.task` the first time
`finger_states(landmarks)`	The geometry from section 5: 21 points → 5 values
`FingerDetector`	Creates the MediaPipe Hand Landmarker (VIDEO mode, 1 hand). `detect(frame)` converts BGR→RGB, adds a timestamp, runs the model and returns `(states, landmarks)`. `draw()` draws the hand skeleton on the frame
`StateStabilizer`	The anti-flicker filter from section 5.3

`FingerDetector` can be used with `with FingerDetector(...) as detector:`. Python then guarantees the model is closed properly, even if an error occurs.

8.3 `mqtt_client.py`: network

`MqttPublisher` wraps the **paho-mqtt** library:

Method	Role
`__init__`	Creates the client, sets the optional login, the **last will** (`0,0,0,0,0`) and the reconnect delays
`start()`	Connects **in the background** (`connect_async` + `loop_start`). The camera starts even if the broker is down
`publish_states(states)`	Turns `(1,0,1,0,0)` into `"1,0,1,0,0"` and publishes it with **retain**
`_on_connect`	Called on every (re)connection. It re-sends the current state so the ESP32 is in sync
`stop()`	Sends `0,0,0,0,0` (LEDs off), waits until it's delivered, then disconnects

8.4 `main.py`: putting it together

```
load settings
open camera            (stop with a clear message if it fails)
start MQTT             (in the background)
create detector + stabilizer
loop:
```

```
    read frame → mirror it
    raw states  = detector.detect(frame)
    states      = stabilizer.update(raw states)
    if states changed: publish them
    if preview on: draw skeleton + finger states + FPS + MQTT status, show window
    quit on Esc / Q
finally (always, even after an error or Ctrl+C):
    release camera, close window, turn LEDs off, disconnect
```

The `finally` block guarantees a clean shutdown: the webcam is released for other apps and the LEDs don't stay on.

8.5 `hand_leds.ino`: ESP32

Part	Role
`LED_PINS = {25, 26, 27, 32, 33}`	Which pin drives which finger's LED
`setup()`	Pins as outputs → LED self-test → Wi-Fi → MQTT server + callback
`loop()`	Reconnect Wi-Fi/MQTT if needed, otherwise `mqtt.loop()`
`connectMqtt()`	Connects with a unique ID, then **subscribes** to the topic
`onMessage()`	Parses and validates the 5 values, then sets the LEDs

9. Real-time behaviour and timing

Approximate delay from a finger movement to the LED change, on a typical laptop:

Step	Typical time
Camera captures the frame	0 to 33 ms (30 FPS)
MediaPipe detection (CPU)	10 to 30 ms
Stabilizer (`STABLE_FRAMES=2`)	about 33 ms extra
MQTT over local Wi-Fi (PC → broker → ESP32)	2 to 20 ms
ESP32 sets the pin	< 1 ms
Total	**roughly 50 to 120 ms**

People generally perceive anything under about 150 ms as "instant". To make it even faster, set `STABLE_FRAMES=1` (accepting a bit more flicker) or `SHOW_PREVIEW=false` (no drawing work).

10. Corrections made to the first version

#	Problem in the first version	Consequence	Fix
1	Used `mediapipe.solutions`, which **no longer exists** in the installed MediaPipe 0.10.35	The program could not start at all (`ImportError`)	Rewritten with the current **MediaPipe Tasks API** (`HandLandmarker`)
2	Thumb test `tip.x < joint.x`	Correct for one hand only. Wrong for the other hand or the back of the hand	Direction-based test that works for both hands (section 5.2)
3	Sent a message **every frame** (~30/s) and printed twice per frame	Network and ESP32 flooded with identical messages, console unreadable	Publish **only on change**
4	`mqtt.Client()` without an API version (paho-mqtt 2.x)	Deprecation warning, and the old callback style is going to be removed	`mqtt.Client(CallbackAPIVersion.VERSION2, ...)`
5	`client.connect()` with no protection	Program crashed if the broker was unreachable	Background connect with automatic reconnect
6	Broker IP hard-coded in the code	You had to edit code to change networks	All settings moved to `.env`
7	No "LEDs off" on exit or crash	LEDs stayed on after the program stopped	Sends `0,0,0,0,0` on exit, plus an MQTT **last will** for crashes
8	No retain	A restarted ESP32 showed nothing until the next finger change	Retained messages + re-sync on reconnect
9	No flicker filtering	LEDs flickered on half-raised fingers	`StateStabilizer`

#	Problem in the first version	Consequence	Fix
10	Everything in one file	Hard to read and change	Split into `config` / `finger_detector` / `mqtt_client` / `main`
11	Broker setup not documented	Mosquitto 2.x **refuses other devices by default**, so the ESP32 could never connect	Documented `listener 1883 0.0.0.0` + firewall rule

> Also check the IP: `.env` uses `192.168.1.197`, but at the time of writing this PC's address was different. Always confirm it with `ipconfig` (see the guide, Part 1.3).

---

11. Limitations and possible improvements

Limitations

- **Hand orientation**: the "tip above knuckle" rule assumes the fingers point **up**. With the hand sideways or upside down, the results are wrong.
- **One hand only** (`num_hands=1`).
- **Lighting and background**: poor light, strong backlight or skin-colored backgrounds reduce accuracy.
- **Thumb**: the sideways test is less reliable when the palm faces sideways rather than toward the camera.
- **Security**: no password or encryption by default (see below).

Improvements

- Use angles between joints instead of up/down comparisons, so detection works in any hand orientation.
- Use MediaPipe's **Gesture Recognizer** to recognise named gestures (👍 , ✌️ , ✊) and trigger actions.
- Use **PWM** on the ESP32 to set LED *brightness* according to how far each finger is bent, instead of just on/off.
- Replace the LEDs with **servo motors** to drive a robotic hand that copies yours.
- Add a phone dashboard (e.g. an MQTT app) that subscribes to the same topic.
- Enable broker authentication and TLS encryption.

---

12. Security note

With `allow_anonymous true`, **anyone on your Wi-Fi network** can connect to the broker and send messages, i.e. turn your LEDs on and off. For a home or lab demo this is acceptable. For anything more serious:

1. Create a password file: `mosquitto_passwd -c C:\mosquitto\passwd myuser`
2. In `mosquitto.conf`, replace `allow_anonymous true` with:

```
allow_anonymous false
password_file C:\mosquitto\passwd
```

3. Set `MQTT_USERNAME` / `MQTT_PASSWORD` in `.env` and `MQTT_USER` / `MQTT_PASS` in `secrets.h`.

Never commit `.env` or `secrets.h`. They are already in `.gitignore`.

---

13. Glossary

Term	Meaning
Arduino IDE	The program used to write C++ code and upload it to boards like the ESP32
BGR / RGB	Two orders for storing a pixel's Blue, Green and Red values
Broker	The MQTT server that receives published messages and delivers them to subscribers
Computer vision	The field of making computers understand images and video
ESP32	A cheap microcontroller with built-in Wi-Fi, by Espressif
FPS	Frames per second
Frame	One single image in a video
GPIO	A pin on the microcontroller that the program can set HIGH (3.3 V) or LOW (0 V)
IoT	"Internet of Things": everyday physical devices connected to a network
IP address	A device's address on the network, e.g. `192.168.1.197`
Landmark	One of the 21 key points MediaPipe finds on a hand
Last Will (LWT)	A message the broker publishes on a client's behalf if it disconnects unexpectedly
LED	Light-Emitting Diode, a small light that only conducts in one direction
Machine learning model	A program that learned a task from examples, instead of being written rule by rule
MediaPipe	Google's library of ready-made vision ML models
Microcontroller	A tiny single-chip computer that runs one program and controls hardware
MQTT	A lightweight publish/subscribe messaging protocol for IoT
Normalized coordinates	Positions from 0.0 to 1.0 relative to the image size, instead of pixels
OpenCV	A library for reading, processing and displaying images and video

Term	Meaning
Payload	The content of an MQTT message (here `"1,0,1,0,0"`)
Pixel	One small colored square of an image
Port	A number identifying a service on a computer (MQTT uses 1883)
Protocol	A set of rules two programs follow to communicate
Publish / Subscribe	Senders publish to a topic; everyone subscribed to it receives the message
QoS	MQTT "Quality of Service": how hard the network tries to deliver a message (0, 1 or 2)
Resistor	A component that limits current; it protects the LED and the pin
Retain	An MQTT flag asking the broker to store the last message and give it to new subscribers
Serial Monitor	An Arduino IDE window showing text printed by the ESP32 over USB
Topic	The "channel name" of MQTT messages, e.g. `hand/gesture`
Tracking	Following a detected object from frame to frame instead of searching the whole image again